形式逻辑：谓词逻辑、产生式系统与不完备性

读前须知

- **前置**：无。这一篇几乎不依赖前面的数学，可以从任何位置插入阅读。
- **这一篇的位置**：符号主义路线的理论根基，也是知识图谱、规则引擎、专家系统的思想来源。
- **读完能做什么**：把一句自然语言写成谓词逻辑表达式；说清产生式系统怎样一步步推理；解释哥德尔不完备定理到底否定了什么。

一、符号主义的知识观

1956 年的达特茅斯会议通常被视为人工智能的起点。与会者中包括约翰·麦卡锡（John McCarthy）、赫伯特·西蒙（Herbert Simon）、马文·明斯基（Marvin Minsky）等后来的图灵奖得主，他们共同的愿景是：

> 让具备抽象思考能力的程序，去解释“由物质合成的机器如何拥有心智”。

把这句偏哲学的话翻译成工程语言：**理想的人工智能应当具备抽象意义上的学习、推理与归纳能力**，其通用性要远远超过“只会下国际象棋”或“只会下围棋”的专用算法。

支撑这个愿景的，是**形式逻辑**。早期研究者的信念可以拆成两步：

1. 人类认知与思维的基本单元是**符号**；
2. 认知过程就是对符号的**逻辑运算**。

于是，人类的抽象思维就可能由计算机中的逻辑门来模拟，从而实现“机械化的人类认知”。反过来，形式逻辑本身也是对智能行为的一种描述：**任何能把某些物理模式或符号转换成另一些模式或符号的系统，都有可能表现出智能。**

1.1 知识表示

要模拟智能，系统首先得有知识。但知识是抽象概念，必须写成机器能理解的形式。

人工智能中常用的知识表示手段有两类：

- **数据结构**：静态地存储问题、问题的中间解答、最终解答，以及解答所依赖的知识；
- **处理算法**：在已有问题与知识之间做动态交互。

两者合起来，才构成完整的知识表示体系。**补充说明**：这个“静态结构 + 动态操作”的二分法，和现代程序设计中“数据 + 算法”的思路完全一致，只是这里的关注点变成了知识能否被机器理性地使用。

二、从三段论到命题逻辑

2.1 三段论

形式逻辑最古老的实例，是古希腊哲学家亚里士多德提出的**三段论（syllogism）**：由两个前提推出一个结论。

一个标准例子，可以按“三句话”来记：

- **大前提**：所有科学都是不断发展的；
- **小前提**：人工智能是一门科学；
- **结论**：所以，人工智能是不断发展的。

> **新手提示：为什么叫“三段论”**
>
> “段”指的是判断，三段就是三个判断：一个一般性的陈述（大前提），一个把具体对象归入其中的陈述（小前提），一个由此推出的陈述（结论）。它的力量不来自内容，而来自**形式**——把“科学”“人工智能”替换成任何别的词，只要形式正确、前提为真，结论就一定为真。

亚里士多德的贡献不在于某个具体结论，而在于确定了一套**形式化推导过程**：只要前提为真、推理形式有效，结论必然为真——**这个过程完全摆脱了具体内容的限制**。这直接催生了符号推理，并深刻影响了后来的数理逻辑。

补充说明：三段论的有效性可以用集合关系来理解。把“科学”看作集合 S，“不断发展的东西”看作集合 D，大前提即 $S \subseteq D$；小前提“人工智能 $\in S$”，于是“人工智能 $\in D$”。这就是谓词逻辑中全称实例化的雏形。

2.2 命题逻辑及其局限

人工智能中最基本的逻辑系统是**谓词逻辑**，而谓词逻辑的一个特例是**命题逻辑（propositional logic）**。

在命题逻辑中，**命题是逻辑处理的基本单位**，只能对整体真伪做出判断，不能再拆。

> **新手提示：“命题”到底指什么样的句子**
>
> 命题必须是**能够判断真假**的陈述句。“雪是白的”“3 大于 5”都是命题（后一个是假命题，但依然是命题）；而“请把门关上”“雪是白的吗”不是命题，因为它们无所谓真假。这是形式逻辑能被机器处理的前提——只有真假两个取值，才谈得上运算。

局限很明显：

- 无法反映命题所描述对象**内部的结构**与逻辑特征；
- 无法体现不同事物之间的共同特征。

举例：单独给出命题“老李是小李的父亲”，在缺少上下文时，既无法确定“老李”“小李”具体所指，也无法据此判断真伪——因为这个命题的结构信息（谁是谁的父亲）被整体打包成了一个不可分析的黑盒。

三、一阶谓词逻辑

为了扩展表达能力，在命题逻辑之上发展出**谓词逻辑（predicate logic）**。它把命题拆成三个部分：

成分	含义	例子
个体词（individual）	可以独立存在的具体或抽象对象	“老李”“小李”
谓词（predicate）	描述个体词的属性或相互关系	“是……的父亲”
量词（quantifier）	描述个体词的数量关系	全称量词 ∀、存在量词 ∃

三者组合起来，就能构成有内部结构的命题。不同的命题之间，还可以用**逻辑联结词（logical connective）** 组合成复合命题。

> **新手提示：∀ 和 ∃ 分别怎么念**
>
> ∀ 读作“对所有的”（for all），∃ 读作“存在至少一个”（there exists）。所以 $\forall x\, P(x)$ 就是“对每一个 x，P(x) 都成立”，$\exists x\, P(x)$ 就是“至少有一个 x 使 P(x) 成立”。注意 ∃ 只保证“有”，不保证“只有一个”；要表达唯一性得用 ∃!，或者在公式里额外加约束。

3.1 五种逻辑联结词（按优先级由高到低）

联结词	符号	复合命题	读法
否定	¬	¬P	非 P
合取	∧	P∧Q	P 且 Q
析取	∨	P∨Q	P 或 Q
蕴涵	→	P→Q	如果 P，那么 Q
等价	↔	P↔Q	P 当且仅当 Q

补充说明：蕴涵的真值规则最容易被误解。P→Q 只在“P 真而 Q 假”时为假，**当 P 为假时整个蕴涵为真**（称为“空真”）。这就是为什么“如果太阳从西边升起，我就请你吃饭”这种话在逻辑上算是真命题。等价则要求双向蕴涵：P↔Q 即 (P→Q) ∧ (Q→P)。

在谓词逻辑中，除了常量符号，还可以使用**变量符号**与**函数符号**。变量与函数的引入，大幅拓展了表达范围，也提升了普适性。

3.2 两类知识

谓词逻辑可以表示两类知识：

- **事实性知识**：事物的概念、状态、属性，通常用析取与合取联结的谓词公式表示；
- **规则性知识**：事物之间确定的因果关系，通常用蕴涵符号连接的谓词公式表示。

3.3 知识表示的三个步骤

一般流程如下：

1. **定义谓词及个体**，明确每个谓词、每个个体的确切含义；
2. **为变量赋值**：根据要表达的事物或概念，给谓词中的变量赋予特定取值；
3. **用联结词组织**：按语义把各个谓词连接成完整公式。

经过这三步，抽象的知识就转化为计算机可识别、可处理的数据结构。

例子：用谓词逻辑表示“所有自然数都是大于零的整数”（这里沿用课程中的约定，把自然数理解为正整数）。

先定义三个谓词：N(x) 表示 x 是自然数，P(x) 表示 x 大于零，I(x) 表示 x 是整数。再按语义连接：

$$(\forall x)\,(N(x) \rightarrow P(x) \wedge I(x))$$

> **新手提示：这个公式怎么念出来**
>
> 从左往右念：“对所有的 x，如果 x 是自然数，那么 x 大于零且 x 是整数。”这里 ∀x 管住的是后面整个括号里的内容，→ 把括号分成“条件”和“结论”两半，∧ 再把两个结论并在一起。

注意这里外层必须是 ∀ 加蕴涵，而不能直接写成 ∀x N(x)——那表达的是“所有个体都是自然数”，语义完全不同。**这类“量词 + 蕴涵”的固定搭配，是写谓词公式最基本的肌肉记忆。**

四、产生式系统与自动推理

用形式逻辑表示知识只是手段，目的是让机器在知识基础上进行**自动推理、归纳与演绎**，从而得出新结论。

就现阶段而言，人类智能与人工智能的主要差距恰恰体现在推理能力上：**人不需要把海量数据全部吞下再学习**，而是从少量数据中提取特征、归纳规律，再以这条一般性规律作为判断依据。一个经典的反例是：在数字图像上叠加一点精心设计的扰动，就能让神经网络把海龟识别成步枪；而这点小把戏骗不过会思考的人。差别不在感知精度，而在推理与常识。

4.1 产生式规则

实现自动推理的基础设施是**产生式系统（production system）**。它用"产生式规则"描述符号串的替换过程，把推理与行为都写成规则，机制上模仿人类认知，因而被早期绝大多数专家系统采用。

产生式规则通常表示因果关系，基本形式为

$$P \to Q$$

- P 称为**规则前件（antecedent）**，可以是单个简单条件，也可以是多个条件经联结词组成的复合条件；
- Q 称为**规则后件（consequent）**，可以是结论，也可以是动作。

当多条规则协同工作时，一条规则的结论可以作为另一条规则的前提，从而解决更复杂的问题——这样构成的整体就是产生式系统。

4.2 三个基本组成

组成	作用	关键点
规则库（rule base）	存储以产生式形式表示的规则集合	系统的核心，规则的完整性、准确性、合理性直接决定性能
事实库（fact base）	存储输入事实、中间结果与最终结果	当某条规则前件与库中事实匹配时，规则被**激活**，结论写入事实库
推理机（inference engine）	控制与协调规则库和事实库运行	包含推理方式与控制策略，决定"先匹配哪条规则"

补充说明：多个规则同时被激活时会出现**冲突（conflict）**，需要"冲突消解策略"决定先执行哪一条，常见做法有优先级、特异性（条件更具体者优先）、新近性等。这一步对系统效率影响很大，早期专家系统的瓶颈往往就在这里。

4.3 三种推理方向

方式	方向	过程	特点
正向推理（forward chaining）	自底向上	从已知事实出发，不断匹配规则前件、得到后件，逐步推出目标结论	数据驱动，适合事实充分、结论未知的场景
反向推理（backward chaining）	自顶向下	从目标假设出发，用规则后件反向匹配，回溯所需前提	目标驱动，适合假设明确、需要验证的场景

方式	方向	过程	特点
双向推理（bidirectional chaining）	双向汇合	同时从事实与目标两端推进，在中间点汇合	效率更高，但控制更复杂

> **新手提示：把正向推理走一遍**
>
> 假设事实库里有“下雨”，规则库里有两条规则：`下雨 → 地面湿`、`地面湿 → 容易滑倒`。推理机先匹配第一条，把“地面湿”写入事实库；再匹配第二条，得到“容易滑倒”。这就是正向推理——**事实像滚雪球一样越滚越多，直到推出目标。**反向推理则倒过来问：“要证明容易滑倒，需要什么？”→ 需要地面湿 → 需要下雨 → 事实库里正好有“下雨”，验证成功。

五、常识：符号系统最深的沟壑

自动推理在数学定理证明上表现强大，可一旦面对日常生活，就远谈不上智能。原因只有四个字：**常识缺失。**

对人类而言，常识是在社会化成长过程中自然形成的。计算机没有这样的成长过程，只能把常识**显式地形式化后灌进内存与硬盘**——这要求把普通成年人的知识与信念完整表达出来，并灵活组织运用。

困难程度远超想象。看一个例子：

> 已知：张三捡起了足球；张三在运动场上。
>
> 问：足球在哪里？

人能瞬间答出“在运动场上”，但对形式系统来说，这需要补充大量隐含公理：被捡起的物体随人移动；人在运动场上则其携带物也在运动场上……类似的隐含前提无穷无尽。至于道德观、世界观这类概念，形式化更是无从下手。

补充说明：这也是后来**知识图谱（knowledge graph）**兴起的原因之一——用实体与关系的大规模结构化网络，部分替代“完备公理系统”的幻想。它不追求逻辑上的完备性，而追求在开放域问答中够用。同一时期，**描述逻辑（description logic）**则在“表达力”与“可判定性”之间寻找平衡，成为本体（ontology）建模的理论基础。

六、哥德尔不完备性定理：绕不过去的边界

讨论人工智能中的形式逻辑，最终都会撞上一个根本问题：**哥德尔不完备性定理。**

6.1 问题的提出

1900 年，德国数学家大卫·希尔伯特在巴黎国际数学家代表大会上提出 20 世纪 23 个最重要的数学问题，其中第二问是：**算术公理系统是否无矛盾。**他期望数学能被建成一座完备且一致的公理大厦。

1931 年，奥地利数学家库尔特·哥德尔（Kurt Gödel）给出了否定的回答——**第一不完备性定理**：

> 在任何足够强（至少能表达初等数论）、自身一致、且公理集可被有效判定（递归可枚举）的形式系统中，都存在一个在该系统内既不能被证明、也不能被证伪的命题。

换句话说，只要系统足够强（能表达算术）、自身不矛盾、且公理集可被有效枚举，就必然存在“不可判定命题”。哥德尔证明的关键，是揭示了公理化系统的阿喀琉斯之踵——**它对自指无能为力。**

6.2 自指的例子

看下面这句话：

> 本命题不可被证明。

这就是一个典型的**自指（self-reference）** 命题。它有两个特点：

1. 讨论的对象不是别的，**正是它自己**——“本命题”指代整个命题；
2. 给出了一个逻辑判断——它不可被证明。

哥德尔证明了：若这个命题可被证明，则它假，系统不一致；若它不可被证明，则它为真，系统不完备。于是**一致性与完备性不可能同时成立。**数学公理化“既完备又自洽”的梦想，就此被戳破。

> **新手提示：为什么“有些命题证不出来”值得震惊**
>
> 按日常直觉，证不出来大概只是“暂时没证出来”，等数学再发展就好了。哥德尔说的却是更狠的一层：**这不是能力问题，而是系统本身注定有盲区。**只要一个系统强到能表达算术，又要保持自身不矛盾，它就必然漏掉某些真命题。换句话说，“完备”和“自洽”这两件好事，一个系统最多只能要一件。

补充说明：还有**第二不完备性定理**——一个足够强的系统无法在自身内部证明自己的一致性（打个比方，就像想揪着自己的鞋带把自己提起来）。这直接击碎了希尔伯特第二问的期待。同类思想还有图灵的**停机问题**：不存在一个通用程序能判断任意程序是否会停机。三者本质相通，都是“系统无法完全把握自身”的不同表述。

6.3 对人工智能意味着什么

不完备性定理的冲击，落在一个基本信念上：**“认知的本质是计算”。**

如果认知确实是计算，那么基于计算机的人工智能若要达到接近人类的思维水平，就必须建立起“自我”的概念；而一旦引入自我，就必然出现**自指**——正好落入不完备性定理的射程。设想计算机在运算中制造出一个代表自身的符号，那么哥德尔构造悖论的手法，就能在机器内部造出既不可证实也不可证伪的“幻境”。

因此，建立在图灵可计算概念之上的**认知可计算主义**研究纲领，显露出根本性的局限。

> **新手提示：符号主义 vs 连接主义**
>
> 这是人工智能的两大路线。**符号主义**主张“智能 = 对符号做逻辑运算”，知识写成规则、推理像做数学证明，专家系统是代表；**连接主义**主张“智能 = 大量简单单元连成网络后涌现”，知识藏在海量参数里，神经网络与深度学习是代表。前者可解释、但难以应对开放世界；后者效果好、但像个黑盒——两条路线的争论至今没有结束。

从学派格局看：依靠神经网络的**连接主义**如日中天，以形式逻辑为依据的**符号主义**则走向衰落。但抛开路线之争，还有两个更值得追问的问题：

1. 人类智能与人工智能的本质差别到底在哪里？
2. 逻辑上不完备，是否等价于“实践上不可用”？——工程系统的目标从来不是绝对完备，而是**在限定领域内足够可靠。**这恰恰解释了为什么今天的模型宁愿放弃可解释的严格推理，转而依赖统计规律。

四个常见误区

- **“哥德尔不完备定理说明人类理性有限”**——它针对的是“足够强、可递归公理化”的形式系统，指出其中必存在既不可证也不可证伪的命题。把它直接推广到人类思维，超出了定理本身的结论。

- **“谓词逻辑既完备又不完备，自相矛盾”**——两个“完备”说的不是一回事：一阶逻辑的完备性指“真命题都能证出来”，哥德尔不完备定理说的是包含算术的系统内部存在不可判定命题，两者层次不同。
- **“蕴含就是因果”**——逻辑蕴含只规定真值关系。前提为假时，整个蕴含式为真，这与现实中的因果毫无关系。
- **“有了概率就没必要学逻辑”**——概率长于处理不确定性，逻辑长于保证推理的严密性。规则系统、知识图谱、程序验证都仍然依赖逻辑。

七、要点回顾

1. 符号主义把认知看作对符号的逻辑运算，形式逻辑是这条路线的基础设施。
2. 知识表示 = 静态数据结构 + 动态处理算法，谓词逻辑是其中最常见的一种形式化手段。
3. 命题逻辑以整体命题为单位，无法刻画内部结构；谓词逻辑把它拆成个体词、谓词、量词。
4. 五种联结词的优先级：否定 > 合取 > 析取 > 蕴涵 > 等价；蕴涵在前件为假时整体为真。
5. 产生式系统由规则库、事实库、推理机组成，推理方向分正向、反向、双向。
6. 符号系统难以处理常识，这是它在开放场景中失效的核心原因。
7. 哥德尔不完备性定理表明：足够强的形式系统必然存在不可判定命题，对“认知即计算”构成理论挑战。
8. 逻辑的不完备不等于工程不可用——限定领域内的可靠性才是工程追求的目标。

八、名词汇总

名称	英文全称	英文缩写	简短介绍
形式逻辑	Formal Logic	—	研究推理形式有效性、与内容无关的逻辑体系
符号主义	Symbolicism	—	认为智能的本质是对符号进行逻辑运算的流派
知识表示	Knowledge Representation	KR	把知识编码为计算机可存储、可推理的形式
三段论	Syllogism	—	由大前提、小前提推出结论的经典演绎推理形式
命题逻辑	Propositional Logic	PL	以整体命题为最小单位、只判断真伪的逻辑系统
谓词逻辑	Predicate Logic	—	把命题拆为个体词、谓词与量词的逻辑系统
一阶逻辑	First-Order Logic	FOL	量词只作用于个体变量、不作用于谓词的谓词逻辑
个体词	Individual Constant	—	命题中独立存在的具体或抽象对象
谓词	Predicate	—	描述个体属性或个体间关系的成分
全称量词	Universal Quantifier	∀	表示“对所有个体都成立”的量词

名称	英文全称	英文缩写	简短介绍
存在量词	Existential Quantifier	∃	表示“至少存在一个个体满足”的量词
逻辑联结词	Logical Connective	—	连接命题以构成复合命题的运算符，含否定、合取、析取、蕴涵、等价
合取	Conjunction	∧	“且”运算，仅当两命题同时为真时为真
析取	Disjunction	∨	“或”运算，至少一个为真时为真
蕴涵	Implication	→	“如果……那么……”运算，仅前件真后件假时为假
等价	Equivalence	↔	双向蕴涵，两命题真值相同时为真
自动推理	Automated Reasoning	—	由机器依据逻辑规则从已知知识推出新结论
产生式系统	Production System	—	由规则库、事实库与推理机组成的规则推理框架
产生式规则	Production Rule	—	形如“若 P 则 Q”的知识表示单元
规则前件	Antecedent	—	产生式规则中触发条件所在的部分
规则后件	Consequent	—	产生式规则中结论或动作所在的部分
规则库	Rule Base	—	存放全部产生式规则的集合，是专家系统的核心
事实库	Fact Base	—	存放输入事实、中间结果与最终结论的数据库
推理机	Inference Engine	—	控制规则与事实匹配、驱动推理过程的程序模块
正向推理	Forward Chaining	—	从已知事实出发、自底向上推导结论的推理方式
反向推理	Backward Chaining	—	从目标假设出发、自顶向下回溯前提的推理方式
双向推理	Bidirectional Chaining	—	同时从事实与目标两端推进、在中间汇合的推理方式
专家系统	Expert System	ES	把领域专家知识写成规则、用于模拟专家决策的系统
知识图谱	Knowledge Graph	KG	以实体—关系三元组组织的大规模结构化知识网络
描述逻辑	Description Logic	DL	兼顾表达力与可判定性的知识表示逻辑体系

名称	英文全称	英文缩写	简短介绍
哥德尔不完备性定理	Gödel's Incompleteness Theorems	—	指出足够强的形式系统必存在不可判定命题、且无法自证一致
自指	Self-Reference	—	命题或系统指向自身的结构，是构造悖论的关键
认知可计算主义	Computationalism	—	认为认知过程本质上等价于可计算过程的立场
停机问题	Halting Problem	—	判断任意程序是否终止的问题，图灵证明其不可判定